

数据结构与算法

第2章 线性表

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2024年3月13日

上节回顾

1、线性表的定义和特点

线性表：由 n 个($n \geq 0$)数据特性相同的元素构成的有限序列称为线性表

2、线性表的顺序表示和实现

顺序存储方法：用一组地址连续的存储单元依次存储线性表的元素，可通过数组 $V[n]$ 来实现。

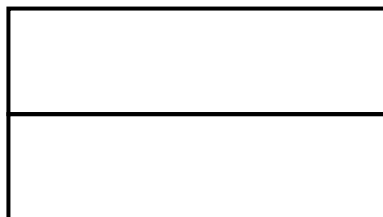
顺序表的结构体数据类型

```
#define MAXSIZE 100    //最大长度
typedef struct {
    ElemType elem[MAXSIZE]; //存放数据元素，数组静态分配
    // ElemType *elem; //存储空间的基地址，数组动态分配
    int length;    //顺序表的当前长度
} SqList; //定义了结构体数据类型SqList，用于表示顺序表
```

SqList L;

Length

elem



整型

地址

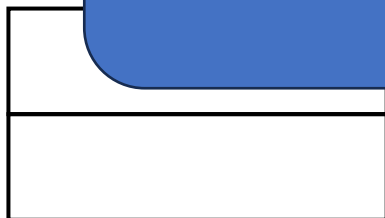
顺序表的结构体数据类型

```
#define MAXSIZE 100    //最大长度
typedef struct {
    ElemType elem[MAXSIZE]; //存放数据元素，数组静态分配
    // ElemType *elem; //存储空间的基地址，数组动态分配
    int length;    //顺序表的当前长度
} SqList; //定义了结构体数据类型SqList 用于表示顺序表
```

SqList L;

SqList *L; //亦可，注意后续
成员引用、形参格式不同

Length
elem



地址

6. 取值 (根据位置i获取相应位置数据元素的内容)

获取线性表L中的某个数据元素的内容

```
int GetElem(SqList L, int i, ElemType &e)
```

```
{
```

```
    if (i<1||i>L.length) return ERROR;
```

```
    //判断i值是否合理，若不合理，返回ERROR
```

```
    e=L.elem[i-1]; //第i-1的单元存储着第i个数据
```

```
    return OK;
```

```
}
```

$O(1)$

随机存取

可直接通过数组下标定位元素

7. 查找

查找的两种情况：

- (1) 根据给定元素的**序号**进行查找(通过**数组下标**定位)
- (2) 根据跟定的**元素值**进行查找

7.查找 (按值查找: 根据指定数据获取数据所在的位置)

查找的基本思想

将给定的元素 e 和顺序表中的每个元素依次进行比较

(1) 若找到与 e 相等的元素, 则**查找成功**, 返回其在表中的“位序”值;

(2) 若找遍整个顺序表, 都没有找到与 e 相等的元素, 则查找失败, 返回值0。

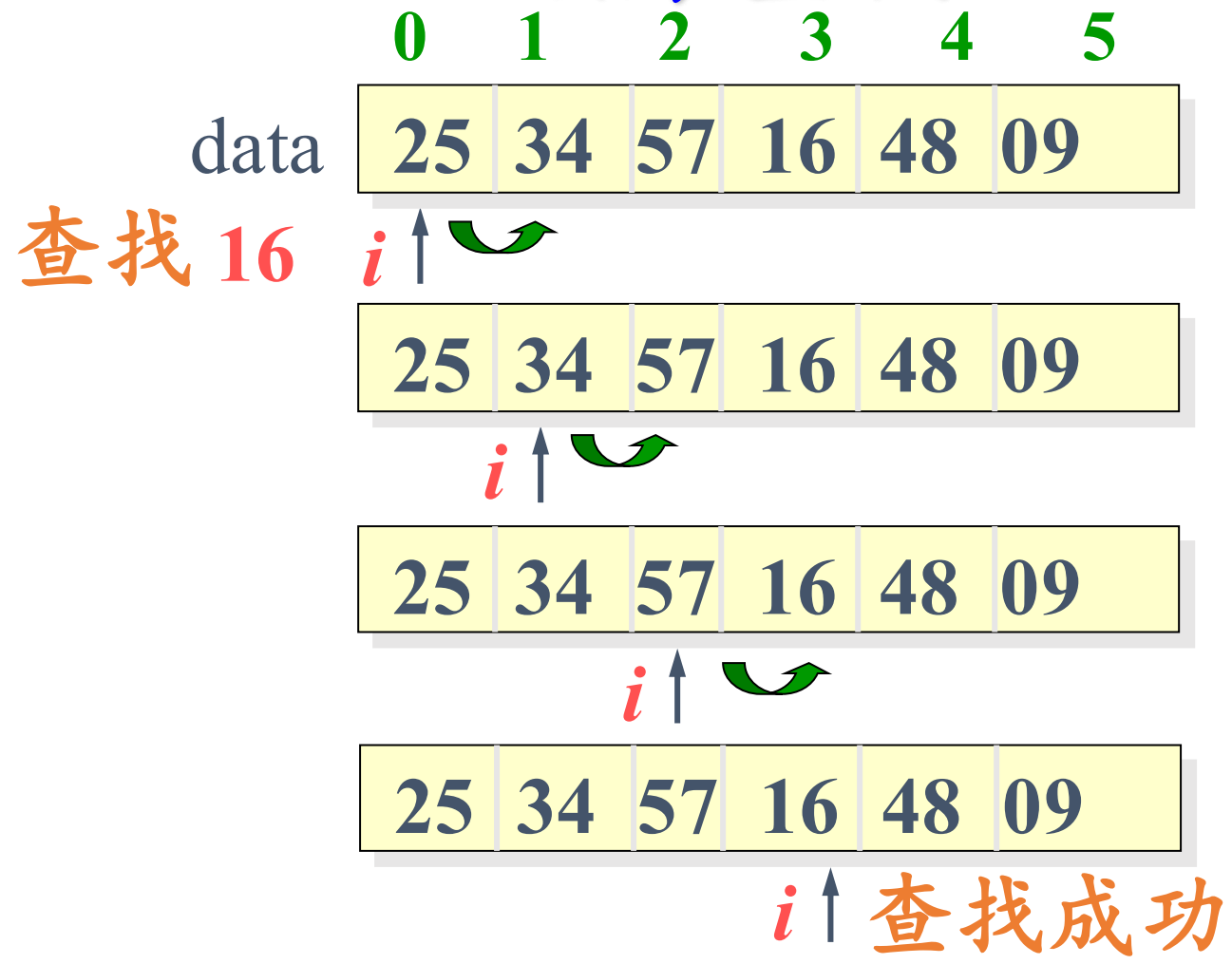


数据下标从0开始, 则表中第 i 个元素对应的下标是 $i-1$

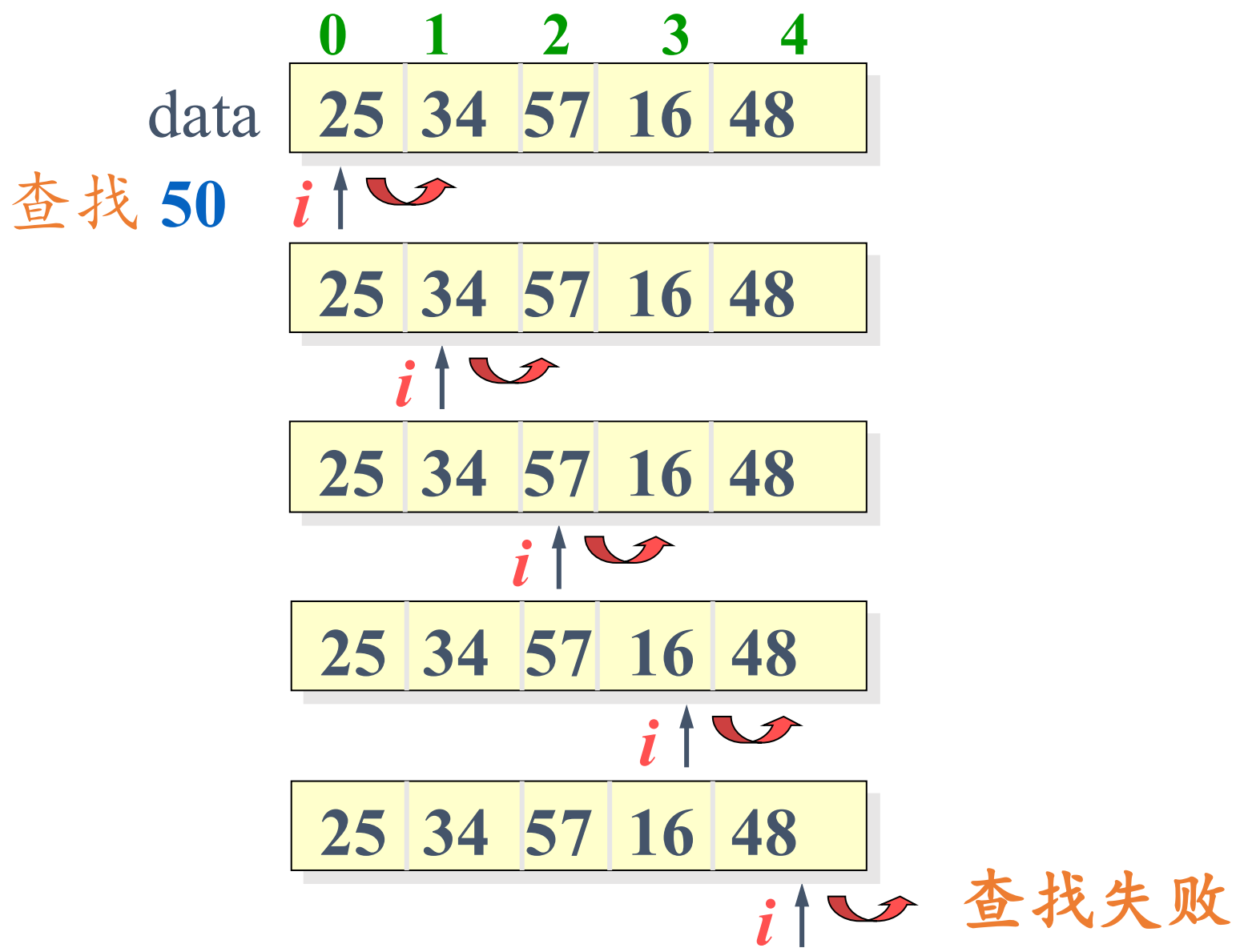
数据下标从1开始使用?

7.查找 (按值查找：根据指定数据获取数据所在的位置)

顺序查找法



7.查找 (按值查找: 根据指定数据获取数据所在的位置)



7.查找 (按值查找: 根据指定数据获取数据所在的位置)

在线性表L中查找值为e的数据元素

```
int LocateElem(SqList L,ElemType e)
```

```
{  
    for (i=0;i< L.length;i++)  
        if (L.elem[i]==e) return i+1;  
    return 0;  
}
```



```
i=0; n=L.length-1;  
While (e!=L.elem[i])  
    i++;  
if (e ==L.elem[i])  
    return i+1;  
else  
    return 0;
```

对不对?

7.查找 (按值查找: 根据指定数据获取数据所在的位置)

在线性表L中查找值为e的数据元素

```
int LocateElem(SqList L,ElemType e)
```

```
{  
    for (i=0;i< L.length;i++)  
        if (L.elem[i]==e) return i+1;  
    return 0;  
}
```



```
i=0; n=L.length-1;  
While (e!=L.elem[i] && i<=n)  
    i++;  
if (e ==L.elem[i] && i<=n)  
    return i+1;  
else  
    return 0;
```

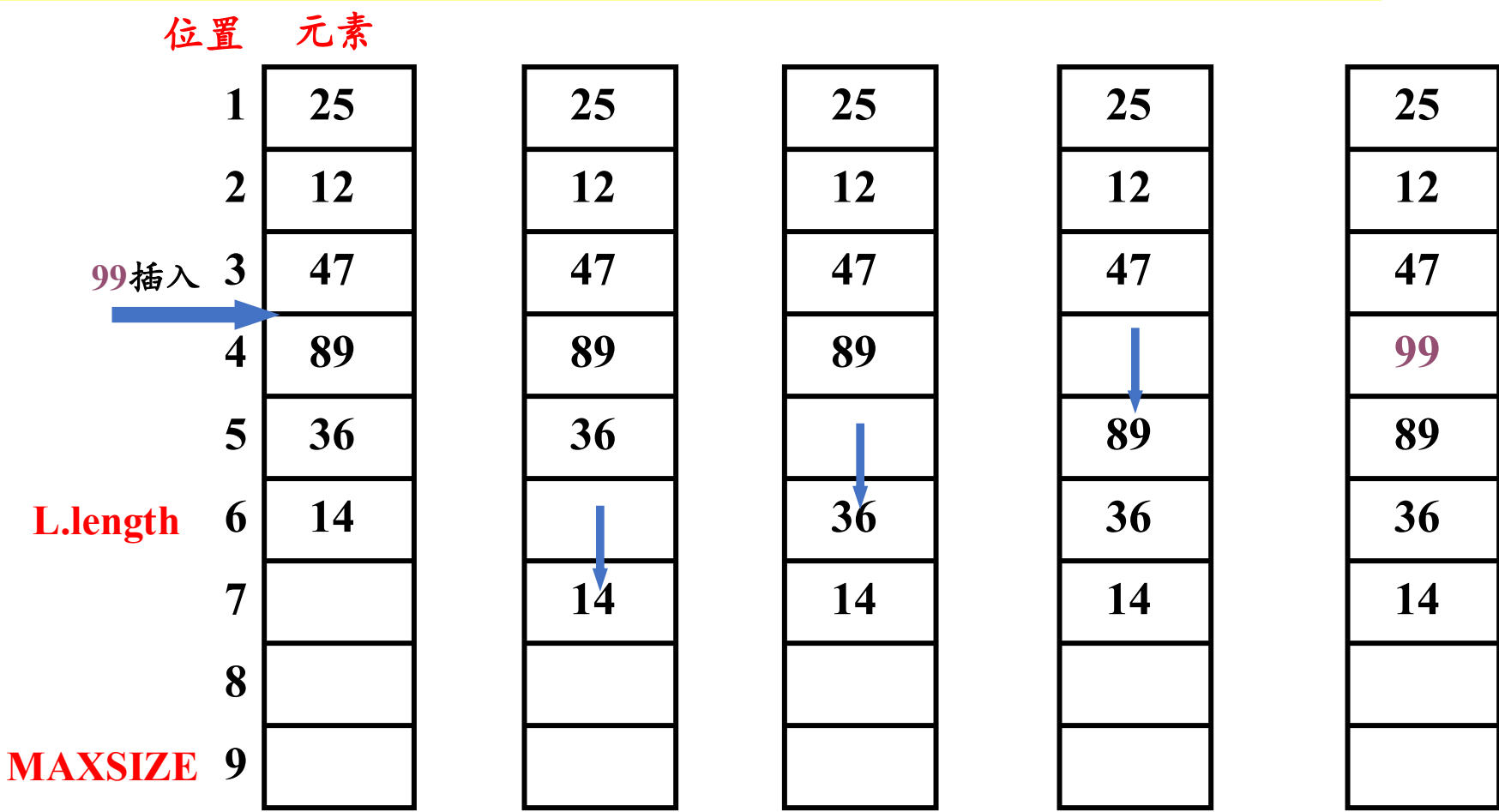
7.查找 (按值查找: 根据指定数据获取数据所在的位置)

在线性表L中查找值为e的数据元素

```
int LocateElem(SqList L,ElemType e)
{
    for (i=0;i< L.length;i++)
        if (L.elem[i]==e) return i+1;
    return 0;
}
```

查找算法时间效率分析 ? ? ?

8. 插入 (插在第 i 个结点之前, $1 \leq i \leq n$)



插第 4 个结点之前, 移动 $6-4+1$ 次

插在第 i 个结点之前, 移动 $n-i+1$ 次

【算法步骤】

- (1) 判断插入位置 i 是否合法。
- (2) 判断顺序表的存储空间是否已满。
- (3) 将第 n 至第 i 位的元素依次向后移动一个位置，空出第 i 个位置。
- (4) 将要插入的新元素 e 放入第 i 个位置。
- (5) 表长加1，插入成功返回OK。

【算法描述】

8. 在线性表L中第i个数据元素之前插入数据元素e

```
Status ListInsert_Sq(SqList &L,int i,ElemType e){  
    if(i<1 || i>L.length+1) return ERROR;           //i值不合法  
    if(L.length==MAXSIZE) return ERROR;              //当前存储空间已满  
    for(j=L.length-1;j>=i-1;j--)  
        L.elem[j+1]=L.elem[j];                       //插入位置及之后的元素后移  
    L.elem[i-1]=e;                                     //将新元素e放入第i个位置  
    ++L.length;                                       //表长增1  
    return OK;  
}
```

注意移动元素时要从最后一个开始往前移，避免覆盖。

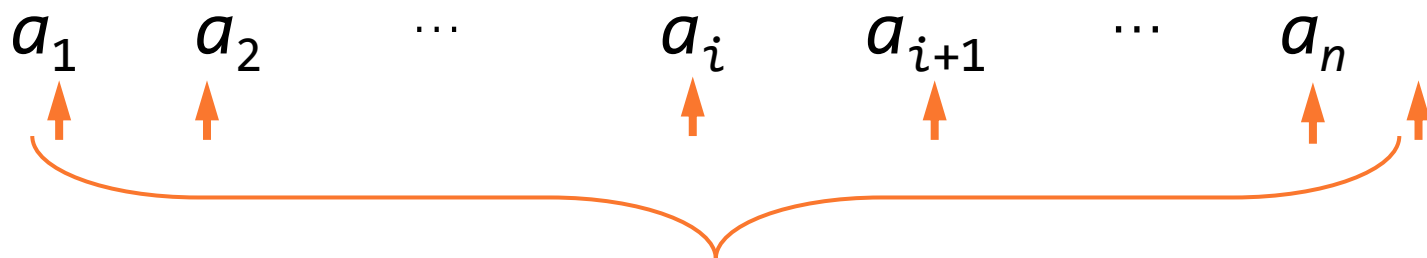
【算法分析】

算法时间主要耗费在移动元素的操作上

- 在线性表L中共有 $n+1$ 个可以插入元素的地方;
- 若插入在尾结点之后($i=n+1$), 则根本无需移动 (特别快);
- 若插入在首结点之前($i=1$), 则表中元素全部后移 (特别慢);
- 若要考虑在各种位置插入 (共 $n+1$ 种可能) 的平均移动次数, 该如何计算?

【算法分析】

平均情况分析：



在线性表L中共有 $n+1$ 个可以插入元素的地方

在插入元素 a_i 时，若为等概率情况，则 $p_i = \frac{1}{n+1}$

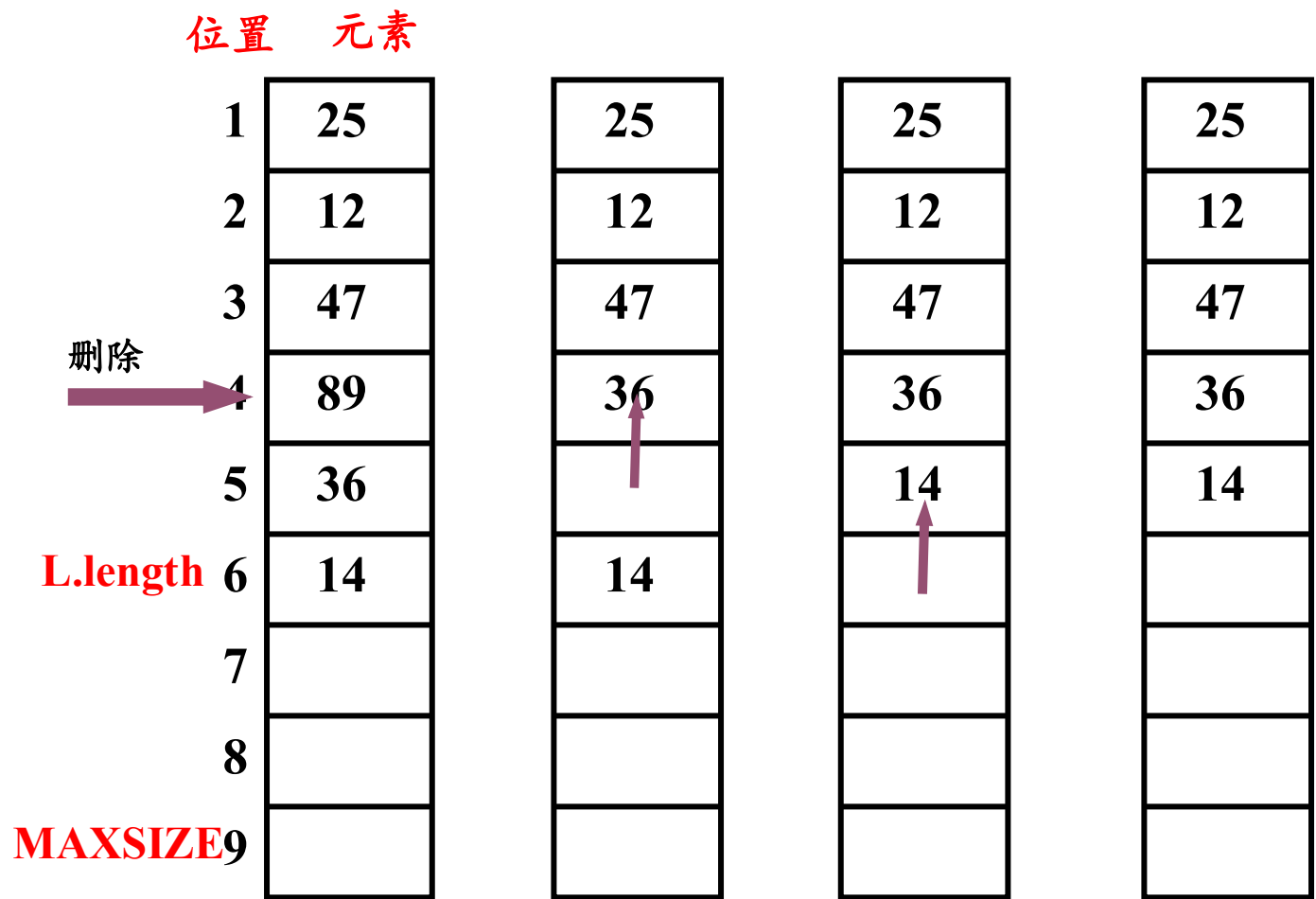
此时需要将 $a_i \sim a_n$ 的元素均后移一个位置，共移动 $n-i+1$ 个元素。

在长度为 n 的线性表中插入一个元素时所需移动元素的平均次数为：

$$\sum_{i=1}^{n+1} p_i (n-i+1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n-i+1) = \frac{n}{2}$$

因此插入算法的平均时间复杂度为 $O(n)$ 。

9. 删除 (删除第 i 个结点)



删除第 4 个结点，移动 **6-4** 次

删除第 i 个结点，移动 **n-i** 次

【算法描述】

将顺序表L中第i个数据元素删除

```
Status ListDelete_Sq(SqList &L,int i){  
    if((i<1)||(i>L.length)) return ERROR;    //i值不合法  
    for (j=i;j<=L.length-1;j++)  
        L.elem[j-1]=L.elem[j];    //被删除元素之后的元素前移  
    --L.length;    //表长减1  
    return OK;  
}
```

- (1) 判断删除位置i是否合法（合法值为 $1 \leq i \leq n$ ）。
- (2) 将欲删除的元素保留在e中。
- (3) 将第i+1至第n位的元素依次向前移动一个位置。
- (4) 表长减1，删除成功返回OK。

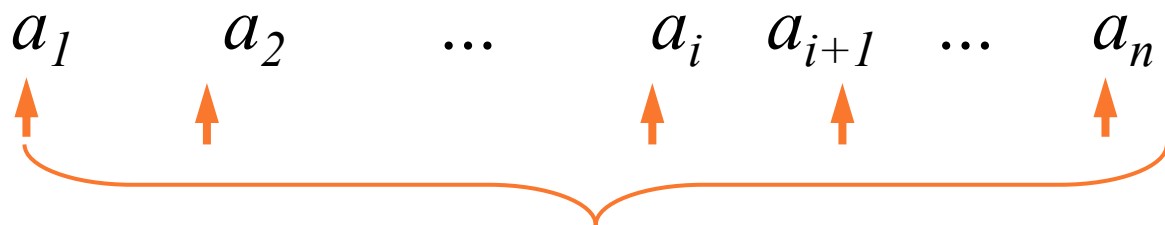
【算法分析】

算法时间主要耗费在移动元素的操作上

- 在线性表L中共有 n 个可以删除元素的地方
- 若删除尾结点，则根本无需移动（特别快）；
- 若删除首结点，则表中 $n-1$ 个元素全部前移（特别慢）；
- 若要考虑在各种位置删除（共 n 种可能）的平均移动次数，该如何计算？

【算法分析】

平均情况分析：



在线性表L中共有 n 个可以删除元素的地方

在删除元素 a_i 时，若为等概率情况，则 $p_i = \frac{1}{n}$

此时需要将 $a_{i+1} \sim a_n$ 的元素均前移一个位置，共移动 $n - (i + 1) + 1 = n - i$ 个元素。

所以在长度为 n 的线性表中删除一个元素时所需移动元素的平均次数为：

$$\sum_{i=1}^n p_i (n - i) = \sum_{i=1}^n \frac{1}{n} (n - i) = \frac{n - 1}{2}$$

因此删除算法的平均时间复杂度为 $O(n)$ 。

顺序表操作算法分析

查找、插入、删除算法的平均时间复杂度为
 $O(n)$

显然，顺序表的空间复杂度 $S(n)=O(1)$
(没有占用辅助空间)

顺序表（顺序存储结构）的特点

- (1) 利用数据元素的存储位置表示线性表中相邻数据元素之间的前后关系，即线性表的**逻辑结构与存储结构一致**
- (2) 在访问线性表时，可以快速地计算出任何一个数据元素的存储地址。因此可以粗略地认为，**访问每个元素所花时间相等**

这种存取元素的方法被称为**随机存取法**



顺序表的优缺点

优点：

- ✓ **存储密度大**（结点数据本身所占存储量/结点结构所占存储量）
- ✓ 可以**随机存取**表中任一元素

缺点：

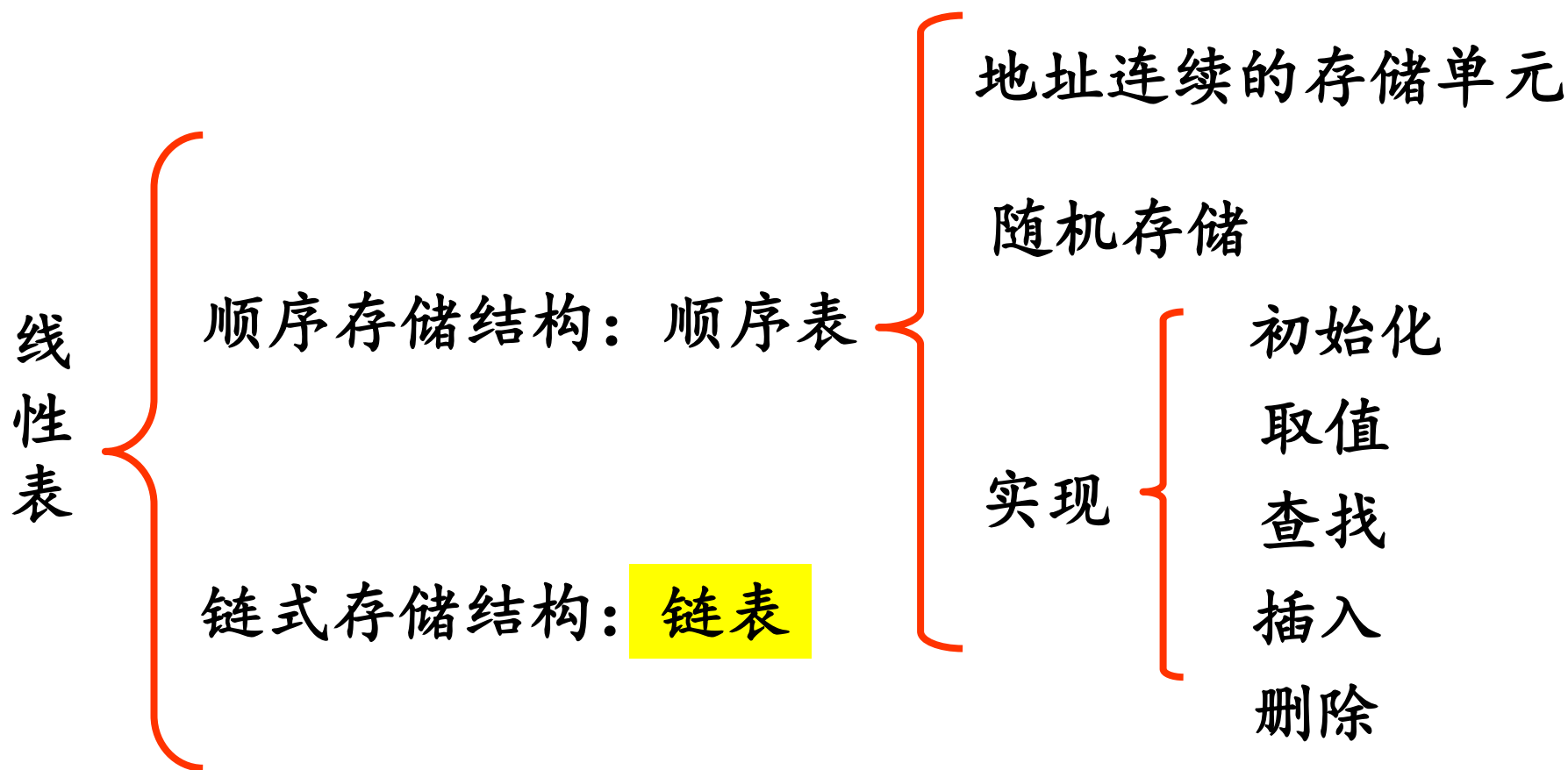
- ✓ 在插入、删除某一元素时，需要移动大量元素
- ✓ 浪费存储空间：“**一次开辟，永久使用**”
- ✓ 属于静态存储形式，数据元素的个数不能自由扩充

为克服这一缺点



链表

线性表 (Linear List) 是数据特性相同的元素构成的有限序列。



2.5 线性表的链式表示和实现

链式存储结构——链表 (Link list)

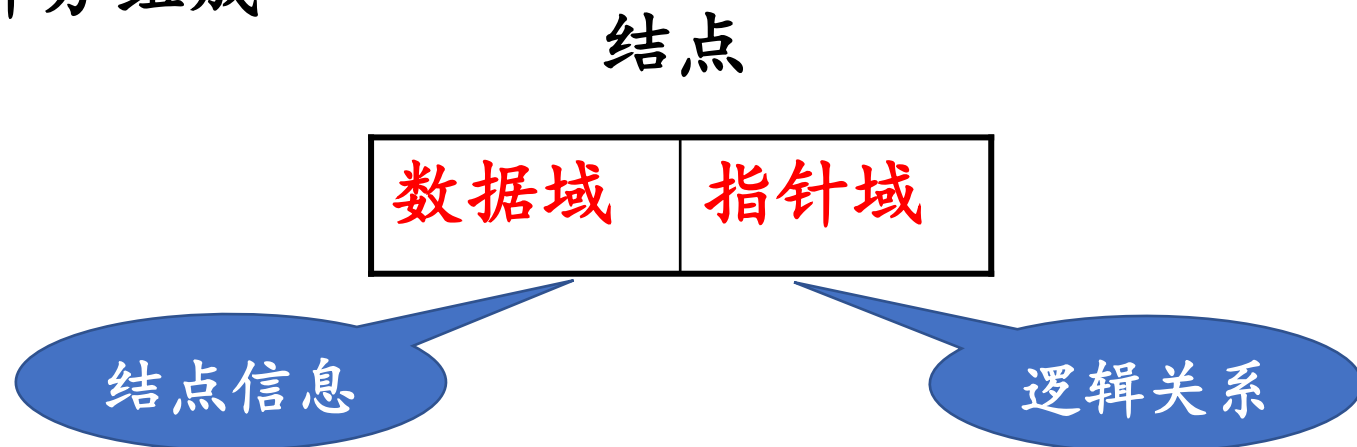
线性表的链式表示又称为**非顺序映像**或**链式映像**。

线性表的每个元素用一个内存结点存储。

结点在存储器中的位置是**任意**的，即逻辑上相邻的数据元素在物理上不一定相邻

与链式存储有关的术语

- 1、**结点**：数据元素的存储映像。由数据域和指针域两部分组成



- 2、**链表**：n 个结点由**指针链**组成一个链表。

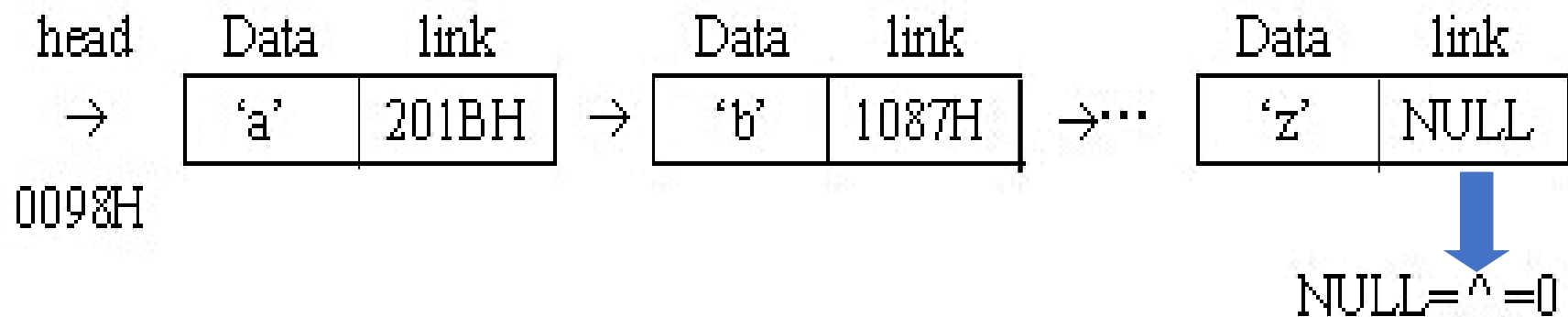
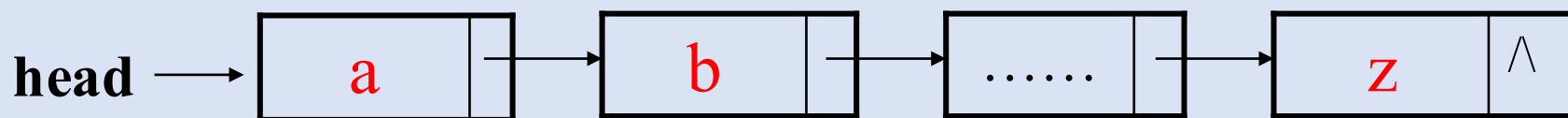
它是线性表的链式存储映像，称为线性表的链式存储结构

链表的逻辑结构虽然是有序的，但物理存储是任意的。

例 画出26个英文字母表的链式存储结构

逻辑结构: (a, b, ..., y, z)

链式存储结构:



若一个结点中的指针域为空，用常量NULL表示

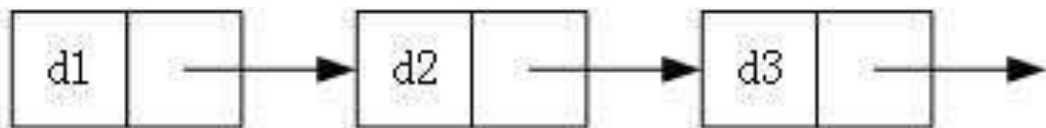
与链式存储有关的术语

3. 链表的分类

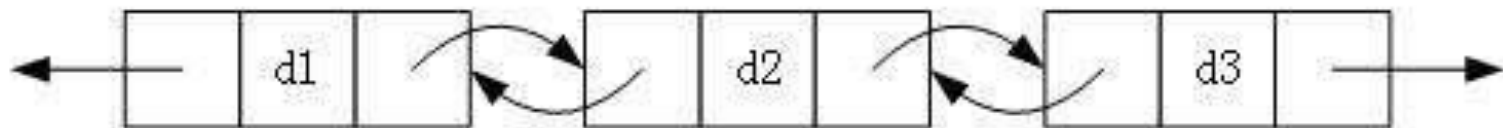
单链表、双链表、循环链表：

- 结点只有一个指针域的链表，称为**单链表**或**线性链表**
- 有两个指针域的链表，称为**双链表**
- 首尾相接的链表称为**循环链表**

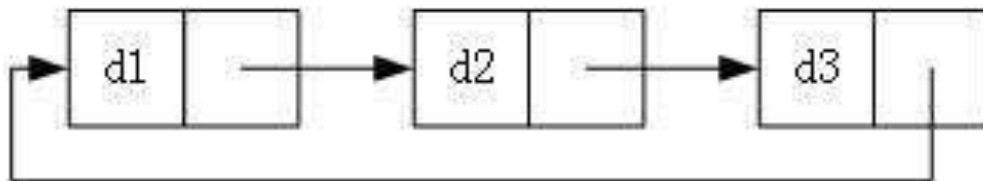
单链表：



双链表：



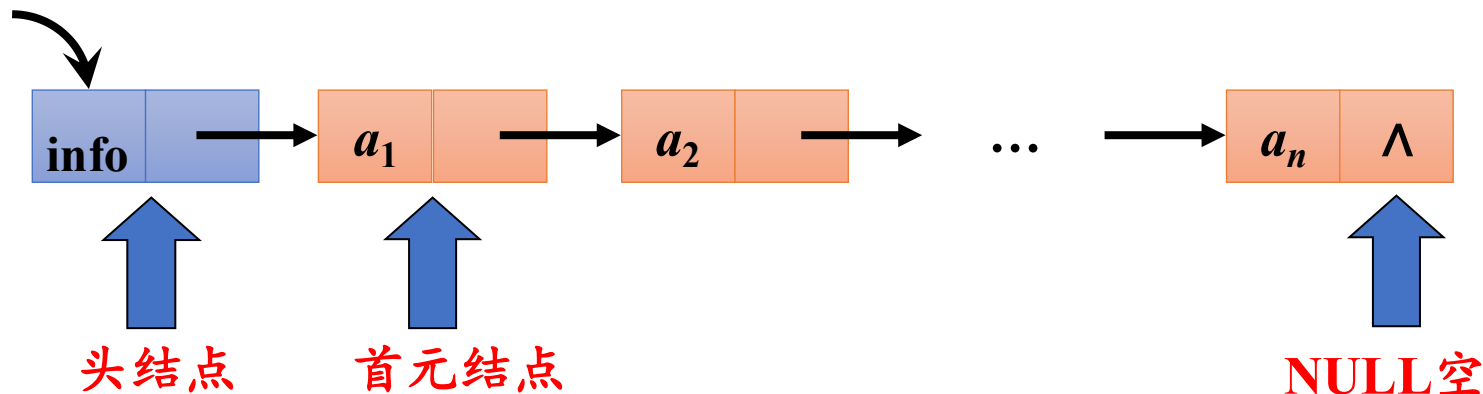
循环单链表：



与链式存储有关的术语

4. 头结点、头指针、首元结点

Head(头指针)



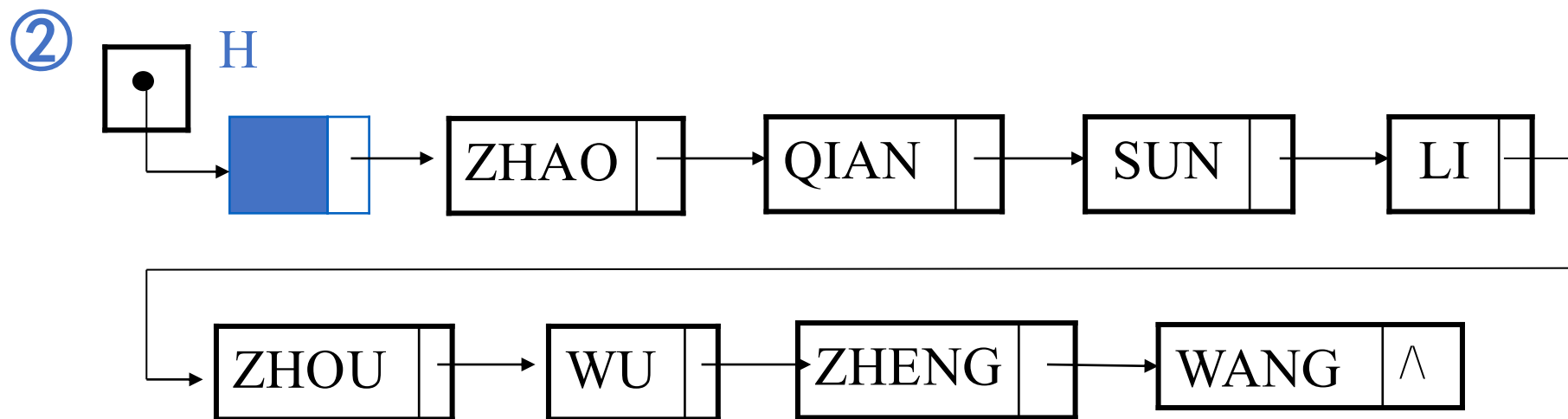
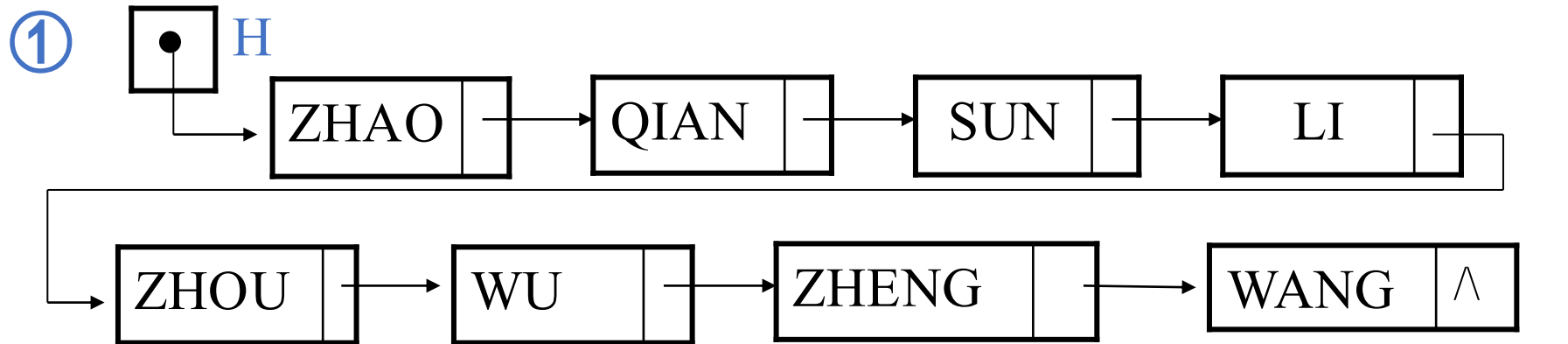
头指针是指向链表中第一个结点的指针

首元结点/首结点是指链表中存储第一个数据元素 a_1 的结点

头结点是在链表的首元结点之前附设的一个结点；数据域内只放空表标志和表长等信息

通常每个链表带有一个头结点，并通过头指针唯一识别该链表

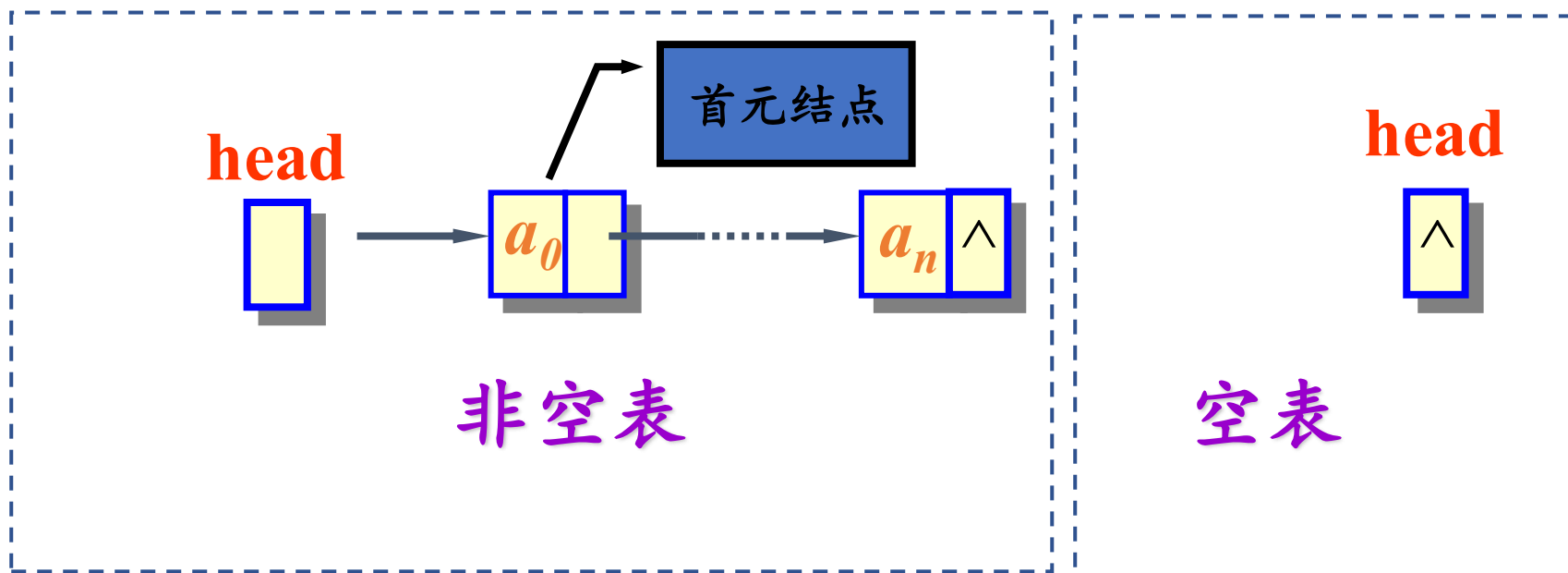
链表的逻辑结构示意图有以下两种形式：



区别：① 无头结点 ② 有头结点

讨论1：如何表示空表？

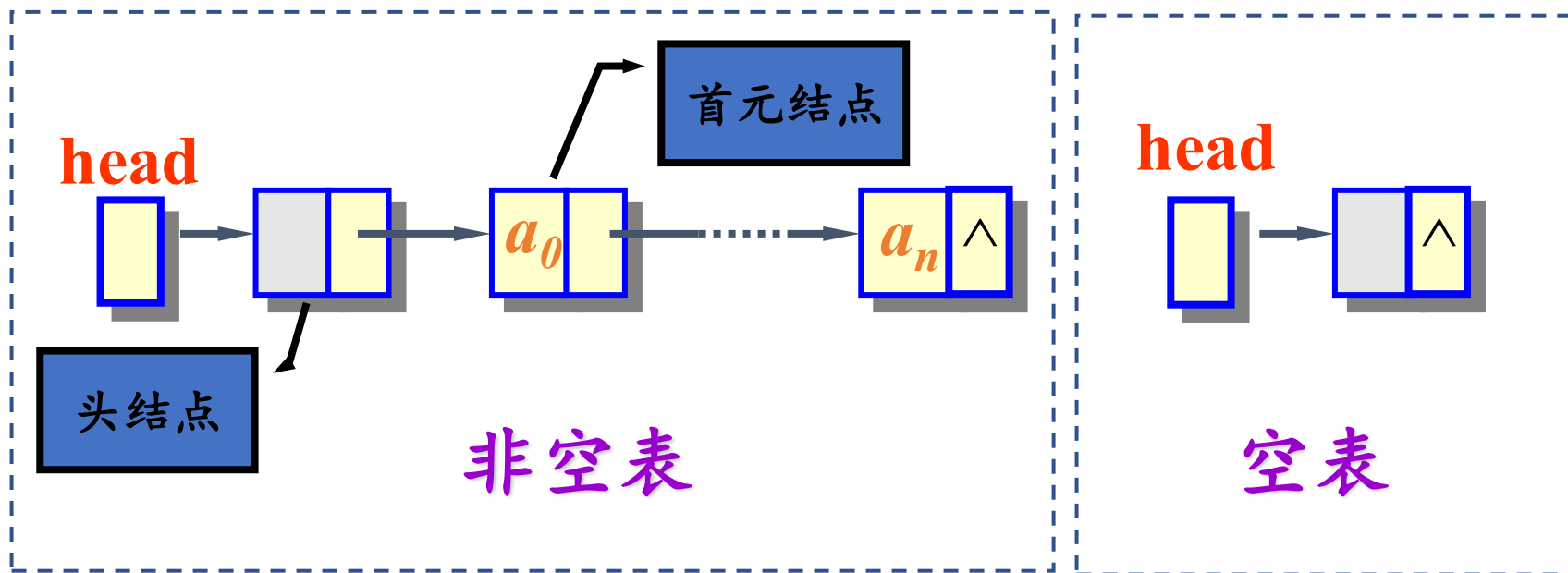
无头结点时，**头指针域为空**时表示空表



head=NULL

讨论1：如何表示空表？

有头结点时，当头结点的指针域为空时表示空表



head->next=NULL

讨论2：在链表中设置头结点有什么好处？

1. 便于首元结点的处理

- 首元结点的地址保存在头结点的指针域中，所以在链表的第一个结点的操作和其他结点一致，无需进行特殊处理；
- 头指针不受影响。

例如，若链表无头结点，则对于在链表中第一个数据结点之前插入一个新结点，或者对链表中第一个数据结点做删除操作，都必须要当做特殊情况，进行特殊考虑；而若链表中设有头结点，以上两种特殊情况都可被视为普通情况，不需要特殊考虑，降低了问题实现的难度。

讨论2：在链表中设置头结点有什么好处？

1. 便于首元结点的处理

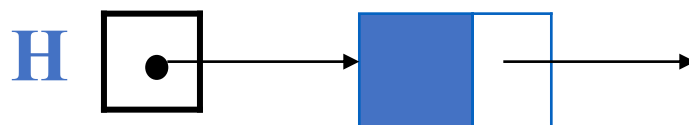
- 首元结点的地址保存在头结点的指针域中，所以在链表的第一个结点的操作和其他结点一致，无需进行特殊处理；
- 头指针不受影响。

2. 便于空表和非空表的统一处理

- 无论链表是否为空，头指针都是指向头结点的非空指针，因此空表和非空表的处理也就统一了。

讨论3. 头结点的**数据域**内装的是什么？

头结点的**数据域**可以为空，也可存放线性表**长度**等附加信息，**但此结点不能计入链表长度值。**



头结点的数据域

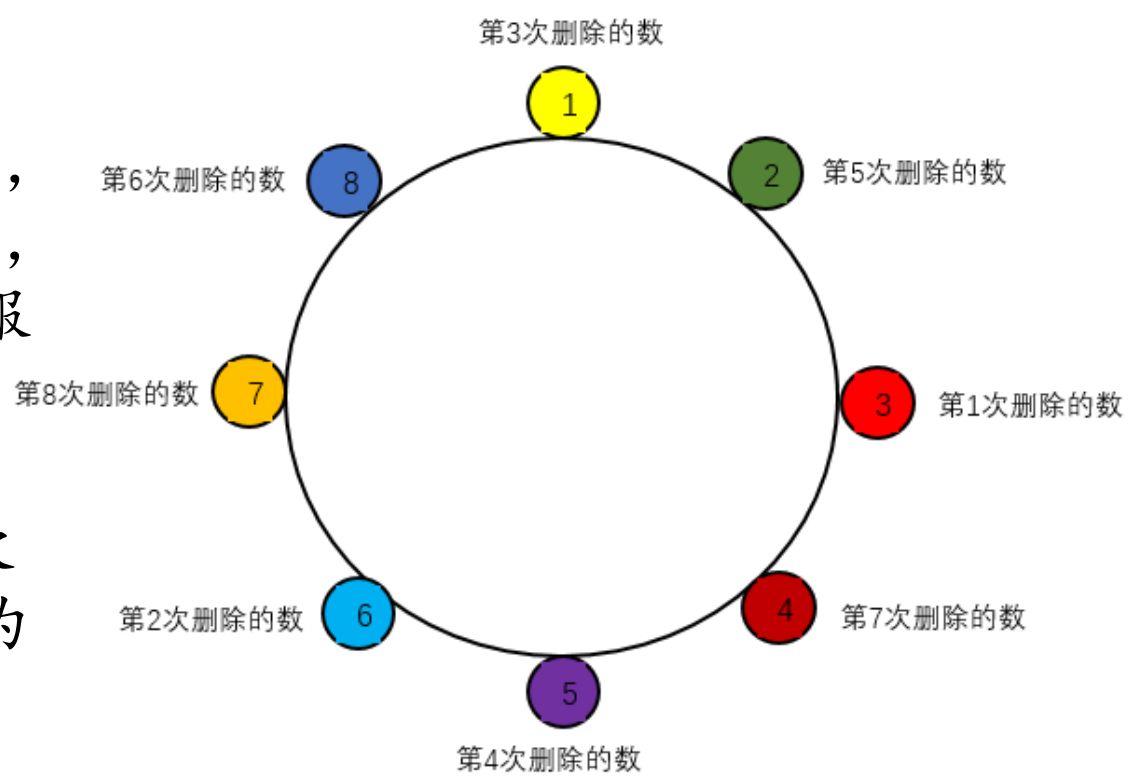
例如，定义链表数据元素是int 类型，头结点的数据域就可以放链表长度

讨论4: 链表是否必须有头结点?

不一定。

约瑟夫问题为：设编号为1，2，.....n的n个人围坐一圈，约定编号为k的人从1开始报数，数到m的那个人出列。它的下一位继续从1开始报数，数到m的人出列，依次类推，直到所有人都出列为止。

循环链表



$$m = 3$$

一个链表可以没有头结点，但不能没有头指针，否则无法访问链表。头指针是链表的必要标识。。

链表（链式存储结构）的特点

(1) 结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理上不一定相邻

(2) 访问时只能通过头指针进入链表，并通过每个结点的指针域向后扫描其余结点，所以寻找第一个结点和最后一个结点所花费的时间不等

这种存取元素的方法被称为顺序存取法

链表的优缺点

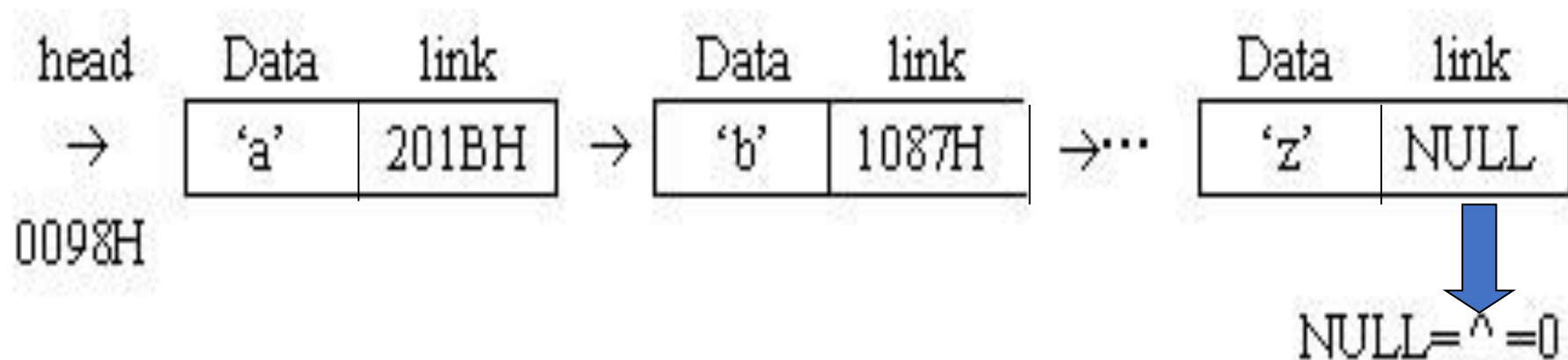
优点

- 数据元素的个数可以自由扩充，存储空间动态分配
- 插入、删除等操作不必移动数据，只需修改链接指针，修改效率较高

链表的优缺点

缺点

- 存储密度小
- 存取效率不高，必须采用**顺序存取**，即存取数据元素时，只能按链表的顺序进行访问（**顺藤摸瓜**）

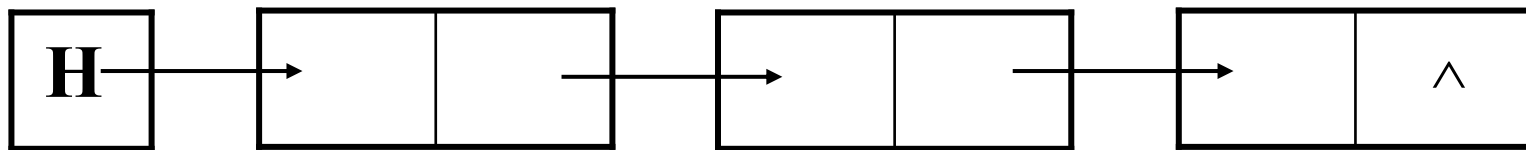


练习

- 1.链表的每个结点中都恰好包含一个指针。
- 2.顺序表结构适宜于进行顺序存取，而链表适宜于进行随机存取。
- 3.顺序存储方式的优点是存储密度大，且插入、删除运算效率高。
- 4.线性表若采用链式存储时，结点之间和结点内部的存储空间都是可以不连续的。
- 5.线性表的每个结点只能是一个简单类型，而链表的每个结点可以是一个复杂类型

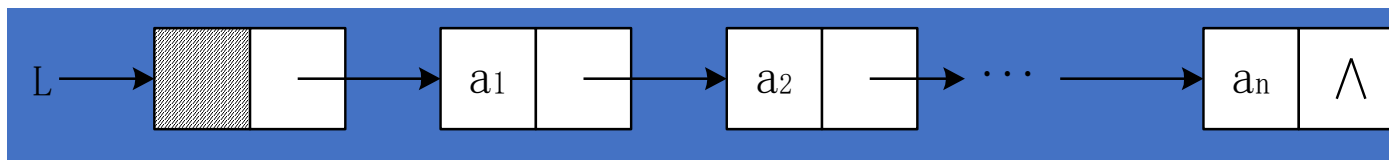
2.5.1 单链表的定义和实现

- 1、链式存储的线性表。
- 2、每个结点中都只含有一个指针域，用来指出后继结点的位置。
- 3、最后一个结点没有指针域，指针域为空（记为NULL或 $\wedge$ ）。
- 4、设置一个表头指针head (常用H或L)，指向链表的第一个结点。

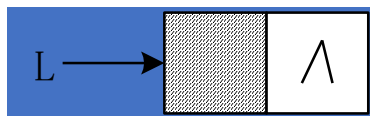


2.5.1 单链表的定义和实现

非空表



空表



带头结点的单链表

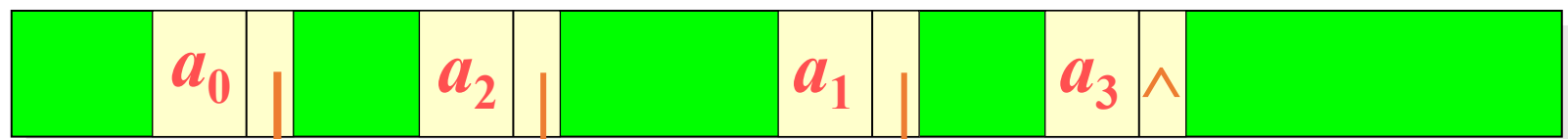
- ✓ 单链表是由表头唯一确定，因此单链表可以用头指针的名字来命名
- ✓ 若头指针名是 L ，则把链表称为表 L

单链表的存储映像



↑
free

(a) 可利用存储空间

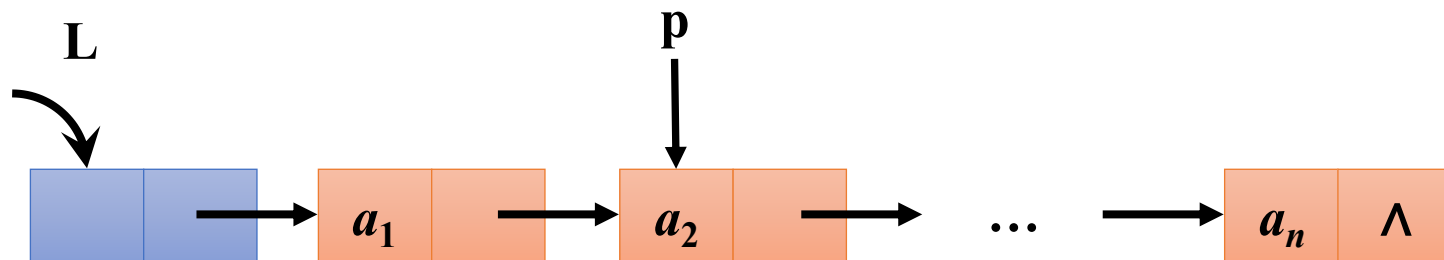


first

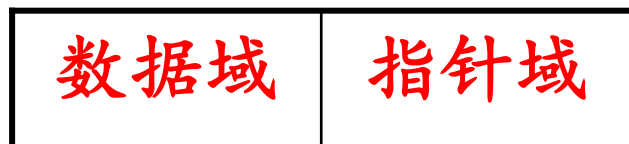
↑
free

(b) 经过一段运行后的单链表结构

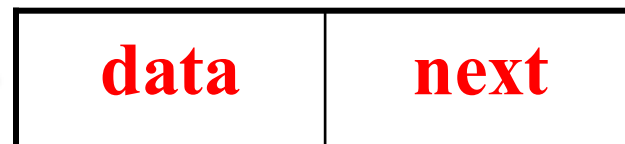
单链表的存储结构定义



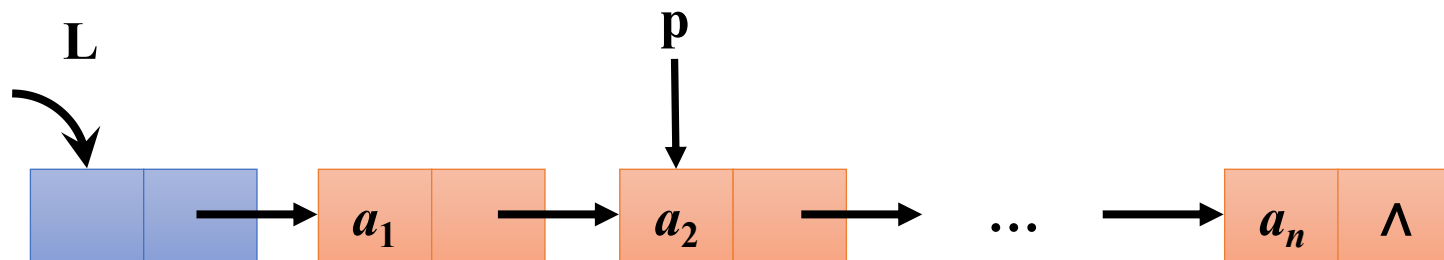
结点



实现——结构体



单链表的存储结构定义



结点

实现——结构体

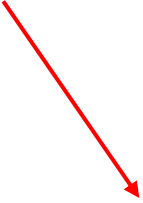


数据类型：
ElemType

存的地址，是
指向哪一种类
型的指针？

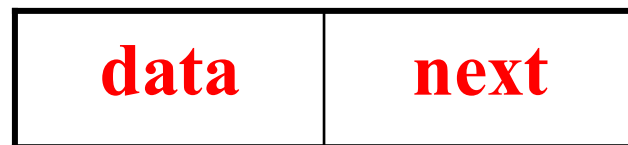
单链表的存储结构定义

```
typedef struct LNode{  
    ElemType data;    //数据域  
    struct LNode *next; //指针域  
}LinkNode; // 单链表结点类型的别名
```



类型别名，不是变量名

实现——结构体



单链表的存储结构定义

这里为什么是
Struct LNode?

```
typedef struct LNode{  
    ElemType data;    //数据域  
    struct LNode *next; //指针域  
}LinkNode; // 单链表结点类型的别名
```

类型别名，不是变量名

实现——结构体

data	next
------	------

单链表的存储结构定义

回顾typedef 用法！

- 是 C 语言中的一个关键字，用于为已有的数据类型定义一个新的名字（别名）
- 基本语法：

typedef 原类型名 新类型名；

例如：

```
typedef int Integer;           // Integer 就是 int 的别名  
Integer a = 10;               // 等价于 int a = 10;
```

单链表的存储结构定义

回顾typedef 用法！

- 给结构体起别名：

```
struct Student {  
    char name[20];  
    int age;  
};  
  
// 使用 struct Student 声明变量：  
struct Student stu1;  
  
// 用 typedef 简化：  
typedef struct Student Student;  
Student stu2;    // 不再需要写 struct 关键字
```

- 将结构体定义和 typedef 合并在一起

```
typedef struct Student {  
    char name[20];  
    int age;  
} Student;    // 这里的 Student 是类型别名，不是变量名
```

单链表的存储结构定义

这里为什么是
Struct LNode?

```
typedef struct LNode{
```

```
    ElemType data;
```

```
    struct LNode *next;
```

```
}LinkNode; // 单链表
```

这里的 struct LNode 是结构体的标签，而 LinkNode 是 typedef 给这个结构体起的别名。

类型别名，不是变量名

实现——结构体

data	next
------	------

单链表的存储结构定义

为什么需要两个名字？

```
typedef struct LNode{  
    ElemType data;    //数据域  
    struct LNode *next; //指针域  
}LinkNode; // 单链
```

类型别名，不

在前面定义顺序表时，我们写到：

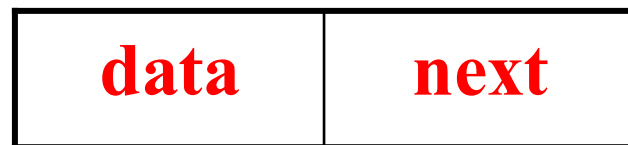
```
typedef struct {  
    ElemType elem[MAXSIZE];  
    int length;  
}SqList;
```

这里是匿名结构体
(但内部无法自引用)

单链表的存储结构定义

```
typedef struct LNode{  
    ElemType  data;    //数据域  
    struct LNode *next; //指针域  
}LinkNode; // 单链表结点类型的别名  
  
LinkNode *L; //定义链表L
```

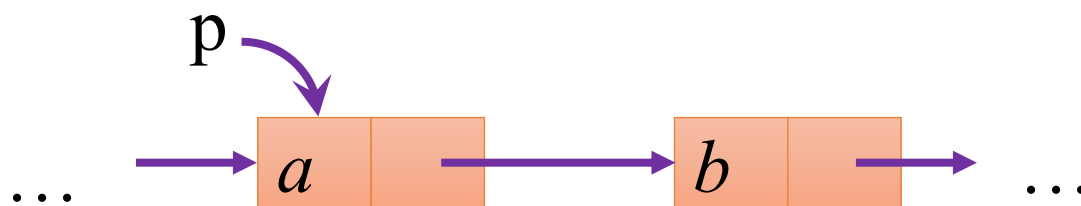
实现——结构体



单链表的存储结构定义

单链表的特点

当访问过一个结点后，只能接着访问它的后继结点，而无法访问它的前驱结点。



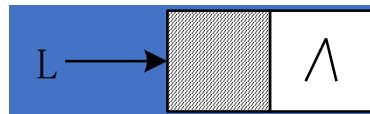
2.5.2 单链表基本操作的实现



1. 初始化、判断空表、销毁、清空、求表长
2. 取值
3. 查找
4. 插入
5. 删除

单链表基本操作的算法实现

- 初始化 (构造一个空表)



【算法思路】

- (1) 生成新结点作头结点，用头指针L指向头结点。
- (2) 头结点的指针域置空。

【算法描述】

```
void InitList (LinkNode *&L){ //构造一个空的链表L
    L=new LinkNode; // L指针头结点, C++
    // L=(LinkNode *)malloc(sizeof(LinkNode)); //C语言
    L->next=NULL; //头结点的指针域为空
}
```